Scala Programming Assignment

Practical Coding Questions — Object-Oriented Programming in Scala

Instructions

- Submit each answer as a separate **.scala** file (e.g., **Q1.scala**, **Q2.scala**, ...). • Each program must be runnable — use **extends App** or a **def main(args: Array[String])** method.
- Include sample output as comments at the bottom of each file.

Q1. Class & Object Basics

Create a class **Student** with fields **name: String**, **rollNo: Int**, and **marks: Array[Int]**. Add a method **percentage()** that returns the average of marks. Create 3 **Student** objects and print each one's name and percentage.

Q2. Constructors

Create a class **Rectangle** with a primary constructor taking **length** and **breadth**. Add an auxiliary constructor that takes a single **side** parameter (for a square) and calls the primary constructor. Add a method **area()**. Instantiate the class using both constructors and print the areas.

Q3. Encapsulation

Create a class **BankAccount** with a private field **balance**. Provide methods **deposit(amount: Double)**, **withdraw(amount: Double)**, and **getBalance(): Double**. Ensure **withdraw** throws an exception (or prints an error) if funds are insufficient. Test by depositing, withdrawing a valid amount, and attempting an invalid withdrawal.

Q4. Inheritance

Create a base class **Animal** with a method **sound(): String** returning "Some generic sound". Create subclasses **Dog**, **Cat**, and **Cow** that override **sound()**. Create a **List[Animal]** containing one of each and print the sound of every animal in a loop.

Q5. Runtime Polymorphism

Create an abstract class **PaymentMethod** with an abstract method **pay(amount: Double): Unit**. Implement subclasses **CreditCard**, **UPI**, and **Cash**, each printing a different confirmation message. Write a function **processPayment(method: PaymentMethod, amount: Double)** that accepts any **PaymentMethod** and calls **pay**. Demonstrate calling it with different subclass instances.

Q6. Traits & Multiple Inheritance

Create two traits: **Printable** (with method **printDetails(): Unit**) and **Discountable** (with method **applyDiscount(price: Double): Double**). Create a class **Product** with fields **name**

and **price** that mixes in both traits and implements their methods. Instantiate a **Product** and call both trait methods.

Q7. Abstract Class vs Trait

Create an abstract class **Employee** with a concrete field **name** and an abstract method **calculateSalary(): Double**. Create a trait **Bonus** with a method **bonusAmount(): Double** returning a fixed bonus. Create a class **Manager** that extends **Employee**, mixes in **Bonus**, and implements **calculateSalary()** to include the bonus. Print the manager's total salary.

Q8. Case Classes & Pattern Matching

Create a sealed trait **Vehicle** with case classes **Car(model: String, seats: Int)**, **Bike(model: String)**, and **Truck(model: String, capacityTons: Double)**. Write a function **describeVehicle(v: Vehicle): String** using pattern matching (**match**/**case**) to return a description specific to each vehicle type. Test it with one instance of each.

Scala OOP Practical Assignment — 8 questions covering classes, constructors, encapsulation, inheritance, polymorphism, traits, abstract classes, case classes, and pattern matching.